Лабораторна робота №3

ПАРАЛЕЛІЗМ ДАНИХ ТА ПАРАЛЕЛІЗМ ЗАДАЧ (Ч. 2)

Виконала студентка групи МІТ-41

Пась Олександра

Хід виконання

1. Створити програму, що використовує для розпаралелювання метод Parallel.For(). Провести ряд обчислювальних експериментів із різним типом елементів масиву (int, double); різною кількістю елементів масиву;різною складністю обчислень

$x = x/10$

$x = x/\pi$

$x = e^{x}/x^{\pi}$

$x = e^{\pi x}/x^{\pi}$

Виміряти час, що витрачається паралельним способом обробки та послідовним. Результати експериментів оформити у вигляді таблиці. Зробити висновки.

```
static double[] arr;
static int[] arrInt;

1 reference
static void Operation1(int index) => arr[index] = arr[index] / 10;
1 reference
static void Operation2(int index) => arr[index] = arr[index] / Math.PI;
1 reference
static void Operation3(int index) => arr[index] = Math.Exp(arr[index]) / Math.Pow(arr[index], Math.PI);
1 reference
static void Operation4(int index) => arr[index] = Math.Exp(Math.PI * arr[index]) / Math.Pow(arr[index], Math.PI);

2 references
static void OpInt1(int index) => arrInt[index] = arrInt[index] / 10;

0 references
static void Main()
{
    Console.OutputEncoding = System.Text.Encoding.UTF8;

    int[] testSizes = { 10000000, 50000000 };

    var doubleCalc = new (string name, Action<int> func)[]
    {
        ("double: /10", Operation1),
        ("double: /π", Operation2),
        ("double: e^x/x^π", Operation3),
        ("double: e^(πx)/x^π", Operation4)
    };

    Console.WriteLine("ДОСЛІДЖЕННЯ ПАРАЛЕЛЬНИХ ОБЧИСЛЕНЬ\n");

    // DOUBLE
    foreach (int length in testSizes)
    {
        Console.WriteLine($"\nМасив double: {length} елементів");
        arr = new double[length];

        for (int i = 0; i < doubleCalc.Length; i++)
        {
            for (int j = 0; j < length; j++) arr[j] = j + 1;

            var timer = new Stopwatch();

            timer.Start();
            for (int j = 0; j < length; j++) doubleCalc[i].func(j);
            timer.Stop();
```

```
            double seqTime = timer.Elapsed.TotalSeconds;

            for (int j = 0; j < length; j++) arr[j] = j + 1;

            timer.Reset();
            timer.Start();
            Parallel.For(0, length, doubleCalc[i].func);
            timer.Stop();
            double parTime = timer.Elapsed.TotalSeconds;

            Console.WriteLine($"{doubleCalc[i].name,-18} посл.: {seqTime,5:F3}с парал.: {parTime,5:F3}с");
        }
    }

    // INT
    foreach (int length in testSizes)
    {
        Console.WriteLine($"\nМасив int: {length} елементів");
        arrInt = new int[length];

        for (int j = 0; j < length; j++) arrInt[j] = j + 1;

        var timer = new Stopwatch();

        timer.Start();
        for (int j = 0; j < length; j++) OpInt1(j);
        timer.Stop();
        double seqTime = timer.Elapsed.TotalSeconds;

        for (int j = 0; j < length; j++) arrInt[j] = j + 1;

        timer.Reset();
        timer.Start();
        Parallel.For(0, length, OpInt1);
        timer.Stop();
        double parTime = timer.Elapsed.TotalSeconds;

        Console.WriteLine($"int: x/10              посл.: {seqTime,5:F3}с парал.: {parTime,5:F3}с");
    }

    Console.ReadKey();
```

рис. 3.1 - код програми

```
ДОСЛІДЖЕННЯ ПАРАЛЕЛЬНИХ ОБЧИСЛЕНЬ


Масив double: 10000000 елементів
double: /10        посл.: 0,211с парал.: 0,189с
double: /π         посл.: 0,215с парал.: 0,072с
double: e^x/x^π    посл.: 2,521с парал.: 2,135с
double: e^(πx)/x^π посл.: 2,477с парал.: 0,540с

Масив double: 50000000 елементів
double: /10        посл.: 0,545с парал.: 0,210с
double: /π         посл.: 0,440с парал.: 0,201с
double: e^x/x^π    посл.: 7,325с парал.: 2,542с
double: e^(πx)/x^π посл.: 7,210с парал.: 2,527с

Масив int: 10000000 елементів
int: x/10              посл.: 0,071с парал.: 0,044с

Масив int: 50000000 елементів
int: x/10              посл.: 0,381с парал.: 0,284с
```

рис. 3.2 - результат виконання

Ця програма порівнює швидкість обчислень у звичайному циклі та паралельно через Parallel.For. Вона створює великі масиви чисел (double і int), виконує над кожним елементом різні математичні операції, і вимірює час виконання. Спочатку обчислення робляться послідовно, потім паралельно, після чого результати виводяться. Мета — показати, що паралельні обчислення можуть працювати швидше, особливо для великих обсягів даних.

2. Модифікувати попередню програму (створивши новий проект у рішенні) таким чином, щоб відбувався вихід з паралельного циклу за умови входження значення елемента у деякий окіл деякого числа (число та відхилення задаються константами).

```
class Program
{
    static double[] array;
    static double targetNumber = 500;
    static double tolerance = 0.5;


    static void ElementHandler(int position, ParallelLoopState loopState)
    {
        array[position] = position * 1.8 + Math.Pow(position, 0.3);

        if (Math.Abs(array[position] - targetNumber) <= tolerance)
        {
            Console.WriteLine($"Збіг! Позиція: {position}, Величина: {array[position]:F3}");
            loopState.Break();
        }
    }

    0 references
    static void Main()
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;
        Console.WriteLine("Пошук елемента в масиві...\n");

        array = new double[10000];

        ParallelLoopResult loopResult = Parallel.For(0, array.Length, ElementHandler);

        if (!loopResult.IsCompleted)
        {
            Console.WriteLine($"\nПроцес зупинено. Номер ітерації: {loopResult.LowestBreakIteration}");
        }
        else
        {
            Console.WriteLine("\nЕлемент не виявлено, обробка завершена.");
        }

        Console.ReadLine();
```

рис. 3.3 - код програми

```
Пошук елемента в масиві...

Збіг! Позиція: 275, Величина: 500,393

Процес зупинено. Номер ітерації: 275
```

рис. 3.4 - результат виконання

Ця програма паралельно заповнює масив і одночасно шукає елемент, значення якого близьке до числа 500. Для кожної позиції обчислюється значення за формулою, і якщо воно відрізняється від 500 не більше ніж на 0.5, програма виводить позицію та зупиняє цикл за допомогою Break(). Після завершення перевіряється, чи було зупинено цикл достроково: якщо так — виводиться номер ітерації, де стався збіг, якщо ні — повідомлення, що елемент не знайдено.

3. Повторити приклади розпаралелювання за допомогою ForEach().

```
class Program
{
    0 references
    static void Main()
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        int size = 10000000;
        var numbers = new List<double>();
        for (int i = 1; i <= size; i++) numbers.Add(i);

        Console.WriteLine("Parallel.ForEach()\n");

        var sw = Stopwatch.StartNew();
        foreach (var x in numbers) { double res = x / 10; }
        sw.Stop();
        Console.WriteLine($"Послідовно: {sw.Elapsed.TotalSeconds:F3}c");

        sw.Restart();
        Parallel.ForEach(numbers, x => { double res = x / 10; });
        sw.Stop();
        Console.WriteLine($"Паралельно: {sw.Elapsed.TotalSeconds:F3}c");
    }
}
```

рис. 3.5 - код програми

```
Parallel.ForEach()

Послідовно: 0,397c
Паралельно: 0,563c
```

рис. 3.6 - результат виконання

Ця програма порівнює швидкість обробки елементів списку звичайним способом і за допомогою Parallel.ForEach. Вона створює список із 10 мільйонів чисел, після чого спочатку проходиться по ньому звичайним циклом foreach і ділить кожен елемент на 10, вимірюючи час. Потім робить те саме, але паралельно, розподіляючи обчислення між потоками. У кінці виводиться час для обох варіантів, щоб побачити різницю в продуктивності.

4. Модифікувати попередню програму (створивши новий проект у рішенні) таким чином, щоб тіло паралельного циклу задавалося лямбда-виразом.

```
class Program
{
    0 references
    static void Main()
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        int size = 10000000;
        var data = new List<double>();
        for (int i = 1; i <= size; i++) data.Add(i);

        Console.WriteLine("Parallel.ForEach() з лямбда-виразом\n");

        // Послідовне виконання
        var sw = Stopwatch.StartNew();
        foreach (var x in data)
        {
            double res = x / 10;
        }
        sw.Stop();
        Console.WriteLine($"Послідовно: {sw.Elapsed.TotalSeconds:F3}с");

        // Паралельне з лямбда-виразом
        sw.Restart();
        Parallel.ForEach(data, x =>
        {
            double res = x / 10;
        });
        sw.Stop();
        Console.WriteLine($"Паралельно (лямбда): {sw.Elapsed.TotalSeconds:F3}с");



        Console.ReadKey();
    }
}
```

рис. 3.7 - код програми

```
Parallel.ForEach() з лямбда-виразом

Послідовно: 0,424с
Паралельно (лямбда): 0,658с
```

рис. 3.8 - результат виконання

Висновок: У ході лабораторної роботи було досліджено принципи паралельних обчислень у C#. Я навчилася використовувати Parallel.For і Parallel.ForEach, порівнювати їх із послідовним виконанням та оцінювати продуктивність. Також було реалізовано дострокове завершення паралельного циклу за умовою. У результаті стало зрозуміло, що паралельні обчислення можуть значно пришвидшити обробку великих обсягів даних, але ефективність залежить від типу задачі.